Part A — Photo Capture and Video Playback App

This app opens the camera, lets you take multiple photos, and when you tap Play Video it encodes those photos into an MP4 file and plays it back on screen. It uses the Camera2 API for capturing and MediaCodec for encoding. Everything is copy-paste ready.

Step 1 — Create the Project

- Open Android Studio → New Project → Empty Views Activity
- Name: PhotoVideoApp
- Package name: com.example.photovideoapp
- Language: Java
- Minimum SDK: API 21
- Click Finish and wait for Gradle sync

Step 2 — app/build.gradle

Replace the entire file with this. Do not change the package name or compileSdk.

File: `app/build.gradle`

```
plugins {
    id 'com.android.application'
}

android {
    namespace 'com.example.photovideoapp'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.photovideoapp"
        minSdk 21
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile(
                'proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
```

```
        targetCompatibility JavaVersion.VERSION_1_8
    }
}

dependencies {
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.11.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'androidx.core:core:1.12.0'
    implementation 'androidx.exifinterface:exifinterface:1.3.6'
}
```

Step 3 — AndroidManifest.xml

Replace the entire file.

File: `app/src/main/AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-feature
        android:name="android.hardware.camera2"
        android:required="true" />

    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="28" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.PhotoVideoApp">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
```

```
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity
            android:name=".VideoPlayerActivity"
            android:exported="false"
            android:screenOrientation="landscape" />

    </application>
</manifest>
```

Step 4 — Resource Files

colors.xml

File: `app/src/main/res/values/colors.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

strings.xml

File: `app/src/main/res/values/strings.xml`

```
<resources>
    <string name="app_name">PhotoVideoApp</string>
</resources>
```

themes.xml

File: `app/src/main/res/values/themes.xml`

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Theme.PhotoVideoApp"
        parent="Theme.MaterialComponents.DayNight.DarkActionBar">
        <item name="colorPrimary">@color/purple_500</item>
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
```

```
        <item name="colorOnSecondary">@color/black</item>
        <item name="android:statusBarColor"
            tools:targetApi="l">?attr/colorPrimaryVariant</item>
    </style>
</resources>
```

Step 5 — Layout Files

5.1 activity_main.xml

File: `app/src/main/res/layout/activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#000000">

    <!-- Camera preview fills most of the screen -->
    <TextureView
        android:id="@+id/textureView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <!-- Bottom control bar -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="#1a1a1a"
        android:padding="12dp">

        <Button
            android:id="@+id/btnCapture"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Capture Photo"
            android:layout_marginEnd="8dp" />

        <Button
            android:id="@+id/btnPlayVideo"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
```

```
            android:text="Play Video"
            android:enabled="false" />

    </LinearLayout>

    <TextView
        android:id="@+id/tvPhotoCount"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Photos taken: 0  (Take at least 2 to play)"
        android:textColor="#ffffff"
        android:textSize="13sp"
        android:gravity="center"
        android:padding="6dp"
        android:background="#111111" />

</LinearLayout>
```

5.2 activity_video_player.xml

File: `app/src/main/res/layout/activity_video_player.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#000000">

    <VideoView
        android:id="@+id/videoView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_centerInParent="true" />

    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Encoding video, please wait..."
        android:textColor="#ffffff"
        android:textSize="16sp"
        android:gravity="center"
        android:padding="16dp"
        android:background="#88000000"
        android:layout_centerInParent="true" />

</RelativeLayout>
```

Step 6 — Java Source Files

Note: All files go into: app/src/main/java/com/example/photovideoapp/

6.1 MainActivity.java

This handles camera preview via Camera2, captures JPEG photos, and passes them to VideoPlayerActivity.

File: `app/src/main/java/com/example/photovideoapp/MainActivity.java`

```
package com.example.photovideoapp;

import android.Manifest;
import android.content.Context;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.graphics.ImageFormat;
import android.graphics.SurfaceTexture;
import android.hardware.camera2.CameraAccessException;
import android.hardware.camera2.CameraCaptureSession;
import android.hardware.camera2.CameraCharacteristics;
import android.hardware.camera2.CameraDevice;
import android.hardware.camera2.CameraManager;
import android.hardware.camera2.CaptureRequest;
import android.hardware.camera2.TotalCaptureResult;
import android.media.Image;
import android.media.ImageReader;
import android.os.Build;
import android.os.Bundle;
import android.os.Handler;
import android.os.HandlerThread;
import android.util.Size;
import android.view.Surface;
import android.view.TextureView;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.ByteBuffer;
```

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class MainActivity extends AppCompatActivity {

    private static final int REQ_CAMERA = 101;

    private TextureView textureView;
    private Button btnCapture, btnPlayVideo;
    private TextView tvPhotoCount;

    private CameraDevice cameraDevice;
    private CameraCaptureSession captureSession;
    private ImageReader imageReader;
    private Handler backgroundHandler;
    private HandlerThread backgroundThread;
    private String cameraId;
    private Size previewSize;

    private final List<String> capturedPhotoPaths = new ArrayList<>();

    // ── Lifecycle ──────────────────────────────────────────────

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        textureView  = findViewById(R.id.textureView);
        btnCapture   = findViewById(R.id.btnCapture);
        btnPlayVideo = findViewById(R.id.btnPlayVideo);
        tvPhotoCount = findViewById(R.id.tvPhotoCount);

        btnCapture.setOnClickListener(v -> capturePhoto());
        btnPlayVideo.setOnClickListener(v -> openVideoPlayer());

        textureView.setSurfaceTextureListener(surfaceTextureListener);
    }

    @Override
    protected void onResume() {
        super.onResume();
        startBackgroundThread();
        if (textureView.isAvailable()) {
            openCamera();
        }
```

```
    }

    @Override
    protected void onPause() {
        closeCamera();
        stopBackgroundThread();
        super.onPause();
    }

    // ── Background Thread ──────────────────────────────────────

    private void startBackgroundThread() {
        backgroundThread = new HandlerThread("CameraBackground");
        backgroundThread.start();
        backgroundHandler = new Handler(backgroundThread.getLooper());
    }

    private void stopBackgroundThread() {
        if (backgroundThread != null) {
            backgroundThread.quitSafely();
            try { backgroundThread.join(); } catch (InterruptedException ignored) {}
            backgroundThread = null;
            backgroundHandler = null;
        }
    }

    // ── Permission ─────────────────────────────────────────────

    private boolean hasCameraPermission() {
        return ContextCompat.checkSelfPermission(this,
            Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED;
    }

    private void requestCameraPermission() {
        ActivityCompat.requestPermissions(this,
            new String[]{ Manifest.permission.CAMERA }, REQ_CAMERA);
    }

    @Override
    public void onRequestPermissionsResult(int code,
            @NonNull String[] perms, @NonNull int[] results) {
        super.onRequestPermissionsResult(code, perms, results);
        if (code == REQ_CAMERA
                && results.length > 0
                && results[0] == PackageManager.PERMISSION_GRANTED) {
            openCamera();
        } else {
```

```
            Toast.makeText(this,
                "Camera permission is required", Toast.LENGTH_LONG).show();
        }
    }

    // ── TextureView Listener ──────────────────────────────────────

    private final TextureView.SurfaceTextureListener surfaceTextureListener =
        new TextureView.SurfaceTextureListener() {
        @Override
        public void onSurfaceTextureAvailable(
                @NonNull SurfaceTexture st, int w, int h) {
            openCamera();
        }
        @Override
        public void onSurfaceTextureSizeChanged(
                @NonNull SurfaceTexture st, int w, int h) {}
        @Override
        public boolean onSurfaceTextureDestroyed(
                @NonNull SurfaceTexture st) { return true; }
        @Override
        public void onSurfaceTextureUpdated(
                @NonNull SurfaceTexture st) {}
    };

    // ── Camera Open / Close ───────────────────────────────────────

    private void openCamera() {
        if (!hasCameraPermission()) {
            requestCameraPermission();
            return;
        }
        CameraManager manager =
            (CameraManager) getSystemService(Context.CAMERA_SERVICE);
        try {
            cameraId = getBackCameraId(manager);
            if (cameraId == null) {
                Toast.makeText(this,
                    "No back camera found", Toast.LENGTH_SHORT).show();
                return;
            }
            CameraCharacteristics chars =
                manager.getCameraCharacteristics(cameraId);
            android.hardware.camera2.params.StreamConfigurationMap map =
                chars.get(CameraCharacteristics
                    .SCALER_STREAM_CONFIGURATION_MAP);
            if (map == null) return;
```

```
            previewSize = chooseSmallerSize(
                map.getOutputSizes(SurfaceTexture.class));

            imageReader = ImageReader.newInstance(
                previewSize.getWidth(), previewSize.getHeight(),
                ImageFormat.JPEG, 2);
            imageReader.setOnImageAvailableListener(
                onImageAvailable, backgroundHandler);

            manager.openCamera(cameraId, cameraStateCallback,
                backgroundHandler);
        } catch (CameraAccessException | SecurityException e) {
            e.printStackTrace();
        }
    }

    private String getBackCameraId(CameraManager manager)
            throws CameraAccessException {
        for (String id : manager.getCameraIdList()) {
            CameraCharacteristics c =
                manager.getCameraCharacteristics(id);
            Integer facing = c.get(CameraCharacteristics.LENS_FACING);
            if (facing != null
                    && facing == CameraCharacteristics.LENS_FACING_BACK)
                return id;
        }
        return null;
    }

    private Size chooseSmallerSize(Size[] sizes) {
        // Pick 640x480 or nearest available for speed
        if (sizes == null || sizes.length == 0)
            return new Size(640, 480);
        Size best = sizes[0];
        for (Size s : sizes) {
            if (s.getWidth() <= 1280 && s.getHeight() <= 720) {
                if (s.getWidth() > best.getWidth()) best = s;
            }
        }
        return best;
    }

    private void closeCamera() {
        if (captureSession != null) {
            captureSession.close(); captureSession = null; }
        if (cameraDevice != null) {
```

```
            cameraDevice.close(); cameraDevice = null; }
        if (imageReader != null) {
            imageReader.close(); imageReader = null; }
    }

    // ── Camera State Callback ──────────────────────────────────

    private final CameraDevice.StateCallback cameraStateCallback =
        new CameraDevice.StateCallback() {
        @Override
        public void onOpened(@NonNull CameraDevice camera) {
            cameraDevice = camera;
            startPreview();
        }
        @Override
        public void onDisconnected(@NonNull CameraDevice camera) {
            camera.close(); cameraDevice = null;
        }
        @Override
        public void onError(@NonNull CameraDevice camera, int error) {
            camera.close(); cameraDevice = null;
            Toast.makeText(MainActivity.this,
                "Camera error: " + error, Toast.LENGTH_SHORT).show();
        }
    };

    // ── Preview ────────────────────────────────────────────────

    private void startPreview() {
        SurfaceTexture st = textureView.getSurfaceTexture();
        if (st == null || cameraDevice == null) return;
        st.setDefaultBufferSize(
            previewSize.getWidth(), previewSize.getHeight());
        Surface previewSurface = new Surface(st);
        try {
            CaptureRequest.Builder builder = cameraDevice
                .createCaptureRequest(CameraDevice.TEMPLATE_PREVIEW);
            builder.addTarget(previewSurface);
            cameraDevice.createCaptureSession(
                Arrays.asList(previewSurface, imageReader.getSurface()),
                new CameraCaptureSession.StateCallback() {
                    @Override
                    public void onConfigured(
                            @NonNull CameraCaptureSession session) {
                        captureSession = session;
                        try {
                            builder.set(CaptureRequest
```

```
                                .CONTROL_AF_MODE,
                                CaptureRequest
                                .CONTROL_AF_MODE_CONTINUOUS_PICTURE);
                            session.setRepeatingRequest(
                                builder.build(), null,
                                backgroundHandler);
                        } catch (CameraAccessException e) {
                            e.printStackTrace();
                        }
                    }
                    @Override
                    public void onConfigureFailed(
                            @NonNull CameraCaptureSession session) {
                        Toast.makeText(MainActivity.this,
                            "Preview config failed",
                            Toast.LENGTH_SHORT).show();
                    }
                }, backgroundHandler);
        } catch (CameraAccessException e) {
            e.printStackTrace();
        }
    }

    // ── Capture Photo ──────────────────────────────────────────

    private void capturePhoto() {
        if (cameraDevice == null || captureSession == null) {
            Toast.makeText(this,
                "Camera not ready", Toast.LENGTH_SHORT).show();
            return;
        }
        try {
            CaptureRequest.Builder captureBuilder = cameraDevice
                .createCaptureRequest(CameraDevice.TEMPLATE_STILL_CAPTURE);
            captureBuilder.addTarget(imageReader.getSurface());
            captureBuilder.set(CaptureRequest.CONTROL_AF_MODE,
                CaptureRequest.CONTROL_AF_MODE_CONTINUOUS_PICTURE);
            captureSession.capture(captureBuilder.build(),
                new CameraCaptureSession.CaptureCallback() {
                    @Override
                    public void onCaptureCompleted(
                            @NonNull CameraCaptureSession session,
                            @NonNull CaptureRequest request,
                            @NonNull TotalCaptureResult result) {
                        // Image saved via ImageReader listener
                    }
                }, backgroundHandler);
```

```
        } catch (CameraAccessException e) {
            e.printStackTrace();
        }
    }

    // ── ImageReader Listener ──────────────────────────────────────

    private final ImageReader.OnImageAvailableListener onImageAvailable =
        reader -> {
            Image image = null;
            try {
                image = reader.acquireLatestImage();
                if (image == null) return;
                ByteBuffer buffer =
                    image.getPlanes()[0].getBuffer();
                byte[] bytes = new byte[buffer.remaining()];
                buffer.get(bytes);
                File photoFile = savePhoto(bytes);
                if (photoFile != null) {
                    capturedPhotoPaths.add(photoFile.getAbsolutePath());
                    runOnUiThread(() -> {
                        int n = capturedPhotoPaths.size();
                        tvPhotoCount.setText(
                            "Photos taken: " + n
                            + (n < 2 ? "  (Take at least 2 to play)"
                                     : "  — Ready!"));
                        btnPlayVideo.setEnabled(n >= 2);
                        Toast.makeText(MainActivity.this,
                            "Photo " + n + " captured",
                            Toast.LENGTH_SHORT).show();
                    });
                }
            } finally {
                if (image != null) image.close();
            }
        };

    private File savePhoto(byte[] data) {
        File dir = getExternalFilesDir("Photos");
        if (dir == null) dir = getFilesDir();
        if (!dir.exists()) dir.mkdirs();
        File file = new File(dir,
            "photo_" + System.currentTimeMillis() + ".jpg");
        try (FileOutputStream fos = new FileOutputStream(file)) {
            fos.write(data);
            return file;
        } catch (IOException e) {
```

```
            e.printStackTrace();
            return null;
        }
    }

    // ── Open Video Player ──────────────────────────────────────

    private void openVideoPlayer() {
        if (capturedPhotoPaths.size() < 2) {
            Toast.makeText(this,
                "Take at least 2 photos first",
                Toast.LENGTH_SHORT).show();
            return;
        }
        Intent intent = new Intent(this, VideoPlayerActivity.class);
        intent.putStringArrayListExtra("photo_paths",
            new ArrayList<>(capturedPhotoPaths));
        startActivity(intent);
    }
}
```

6.2 VideoEncoder.java

This class encodes a list of JPEG files into an MP4 using MediaCodec. It runs on a background thread and calls back when done.

File: `app/src/main/java/com/example/photovideoapp/VideoEncoder.java`

```
package com.example.photovideoapp;

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Canvas;
import android.graphics.Color;
import android.media.MediaCodec;
import android.media.MediaCodecInfo;
import android.media.MediaFormat;
import android.media.MediaMuxer;
import android.util.Log;

import java.io.File;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.util.List;

public class VideoEncoder {

    private static final String TAG    = "VideoEncoder";
```

```
    private static final String MIME   = "video/avc";
    private static final int    WIDTH  = 640;
    private static final int    HEIGHT = 480;
    private static final int    FPS    = 1;   // 1 frame per photo
    private static final int    IFRAME = 1;   // every frame is I-frame
    private static final int    BPS    = 2_000_000;
    // Each photo shows for 2 seconds
    private static final long   FRAME_DURATION_US = 2_000_000L;

    public interface Callback {
        void onSuccess(File outputFile);
        void onError(String message);
    }

    public static void encode(
            List<String> photoPaths,
            File outputFile,
            Callback callback) {
        new Thread(() -> {
            try {
                doEncode(photoPaths, outputFile);
                callback.onSuccess(outputFile);
            } catch (Exception e) {
                Log.e(TAG, "Encoding failed", e);
                callback.onError(e.getMessage());
            }
        }, "VideoEncoderThread").start();
    }

    private static void doEncode(
            List<String> paths, File out) throws IOException {

        MediaFormat format = MediaFormat.createVideoFormat(
            MIME, WIDTH, HEIGHT);
        format.setInteger(MediaFormat.KEY_COLOR_FORMAT,
            MediaCodecInfo.CodecCapabilities.COLOR_FormatYUV420Flexible);
        format.setInteger(MediaFormat.KEY_BIT_RATE,  BPS);
        format.setInteger(MediaFormat.KEY_FRAME_RATE, FPS);
        format.setInteger(MediaFormat.KEY_I_FRAME_INTERVAL, IFRAME);

        MediaCodec encoder = MediaCodec.createEncoderByType(MIME);
        encoder.configure(format, null, null,
            MediaCodec.CONFIGURE_FLAG_ENCODE);
        encoder.start();

        MediaMuxer muxer = new MediaMuxer(
            out.getAbsolutePath(),
```

```
            MediaMuxer.OutputFormat.MUXER_OUTPUT_MPEG_4);
        int trackIndex = -1;
        boolean muxerStarted = false;

        MediaCodec.BufferInfo info = new MediaCodec.BufferInfo();
        long presentationUs = 0;

        for (String path : paths) {
            Bitmap bmp = loadAndScale(path);
            byte[] yuv = bitmapToYuv420(bmp);
            bmp.recycle();

            // Feed input
            int inIdx = encoder.dequeueInputBuffer(10_000);
            if (inIdx >= 0) {
                ByteBuffer buf = encoder.getInputBuffer(inIdx);
                buf.clear();
                buf.put(yuv);
                encoder.queueInputBuffer(
                    inIdx, 0, yuv.length, presentationUs, 0);
                presentationUs += FRAME_DURATION_US;
            }

            // Drain output
            drainEncoder(encoder, muxer, info,
                new int[]{ trackIndex },
                new boolean[]{ muxerStarted },
                false);
            trackIndex    = getTrackIndex(encoder, muxer, info,
                new int[]{ trackIndex },
                new boolean[]{ muxerStarted });
            muxerStarted  = trackIndex >= 0;
        }

        // Signal end of stream
        int inIdx = encoder.dequeueInputBuffer(10_000);
        if (inIdx >= 0) {
            encoder.queueInputBuffer(inIdx, 0, 0, presentationUs,
                MediaCodec.BUFFER_FLAG_END_OF_STREAM);
        }
        drainUntilDone(encoder, muxer, info,
            new int[]{ trackIndex },
            new boolean[]{ muxerStarted });

        encoder.stop();
        encoder.release();
        muxer.stop();
```

```
        muxer.release();
    }

    // —— Simplified drain helper ————————————————————————————————

    private static int getTrackIndex(
            MediaCodec enc, MediaMuxer muxer,
            MediaCodec.BufferInfo info,
            int[] trackRef, boolean[] startedRef) {
        if (startedRef[0]) return trackRef[0];
        int outIdx = enc.dequeueOutputBuffer(info, 0);
        if (outIdx == MediaCodec.INFO_OUTPUT_FORMAT_CHANGED) {
            int t = muxer.addTrack(enc.getOutputFormat());
            muxer.start();
            trackRef[0]   = t;
            startedRef[0] = true;
            return t;
        }
        if (outIdx >= 0) enc.releaseOutputBuffer(outIdx, false);
        return trackRef[0];
    }

    private static void drainEncoder(
            MediaCodec enc, MediaMuxer muxer,
            MediaCodec.BufferInfo info,
            int[] trackRef, boolean[] startedRef,
            boolean endOfStream) {
        int maxTries = 30;
        while (maxTries-- > 0) {
            int outIdx = enc.dequeueOutputBuffer(info, 5_000);
            if (outIdx == MediaCodec.INFO_TRY_AGAIN_LATER) {
                if (!endOfStream) break;
            } else if (outIdx ==
                    MediaCodec.INFO_OUTPUT_FORMAT_CHANGED) {
                if (!startedRef[0]) {
                    trackRef[0] = muxer.addTrack(enc.getOutputFormat());
                    muxer.start();
                    startedRef[0] = true;
                }
            } else if (outIdx >= 0) {
                ByteBuffer buf = enc.getOutputBuffer(outIdx);
                if (buf != null
                        && startedRef[0]
                        && info.size > 0
                        && (info.flags &
                            MediaCodec.BUFFER_FLAG_CODEC_CONFIG) == 0) {
                    buf.position(info.offset);
```

```
                    buf.limit(info.offset + info.size);
                    muxer.writeSampleData(trackRef[0], buf, info);
                }
                enc.releaseOutputBuffer(outIdx, false);
                if ((info.flags &
                        MediaCodec.BUFFER_FLAG_END_OF_STREAM) != 0)
                    break;
            }
        }
    }

    private static void drainUntilDone(
            MediaCodec enc, MediaMuxer muxer,
            MediaCodec.BufferInfo info,
            int[] trackRef, boolean[] startedRef) {
        drainEncoder(enc, muxer, info, trackRef, startedRef, true);
    }

    // —— Bitmap helpers ————————————————————————————————————————

    private static Bitmap loadAndScale(String path) {
        BitmapFactory.Options opts = new BitmapFactory.Options();
        opts.inJustDecodeBounds = true;
        BitmapFactory.decodeFile(path, opts);
        opts.inSampleSize = calculateInSampleSize(
            opts, WIDTH, HEIGHT);
        opts.inJustDecodeBounds = false;
        Bitmap raw = BitmapFactory.decodeFile(path, opts);
        if (raw == null) {
            raw = Bitmap.createBitmap(WIDTH, HEIGHT,
                Bitmap.Config.ARGB_8888);
            new Canvas(raw).drawColor(Color.BLACK);
        }
        return Bitmap.createScaledBitmap(raw, WIDTH, HEIGHT, true);
    }

    private static int calculateInSampleSize(
            BitmapFactory.Options opts, int reqW, int reqH) {
        int h = opts.outHeight, w = opts.outWidth, ss = 1;
        if (h > reqH || w > reqW) {
            int hh = h / 2, hw = w / 2;
            while (hh / ss >= reqH && hw / ss >= reqW) ss *= 2;
        }
        return ss;
    }

    private static byte[] bitmapToYuv420(Bitmap bmp) {
```

```
        int w = bmp.getWidth(), h = bmp.getHeight();
        int[] argb = new int[w * h];
        bmp.getPixels(argb, 0, w, 0, 0, w, h);

        byte[] yuv = new byte[w * h * 3 / 2];
        int yIdx = 0, uvIdx = w * h;

        for (int j = 0; j < h; j++) {
            for (int i = 0; i < w; i++) {
                int p = argb[j * w + i];
                int r = (p >> 16) & 0xff;
                int g = (p >>  8) & 0xff;
                int b =  p        & 0xff;
                int Y  =  ((66*r + 129*g +  25*b + 128) >> 8) + 16;
                int Cb = ((-38*r -  74*g + 112*b + 128) >> 8) + 128;
                int Cr = ((112*r -  94*g -  18*b + 128) >> 8) + 128;
                yuv[yIdx++] = (byte) Math.max(0, Math.min(255, Y));
                if (j % 2 == 0 && i % 2 == 0) {
                    yuv[uvIdx++] = (byte) Math.max(0, Math.min(255, Cb));
                    yuv[uvIdx++] = (byte) Math.max(0, Math.min(255, Cr));
                }
            }
        }
        return yuv;
    }
}
```

6.3 VideoPlayerActivity.java

Receives the photo paths, calls VideoEncoder, then plays the resulting MP4.

File: `app/src/main/java/com/example/photovideoapp/VideoPlayerActivity.java`

```
package com.example.photovideoapp;

import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.MediaController;
import android.widget.TextView;
import android.widget.Toast;
import android.widget.VideoView;

import androidx.appcompat.app.AppCompatActivity;

import java.io.File;
import java.util.ArrayList;
```

```java
public class VideoPlayerActivity extends AppCompatActivity {

    private VideoView videoView;
    private TextView tvStatus;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_video_player);

        videoView = findViewById(R.id.videoView);
        tvStatus  = findViewById(R.id.tvStatus);

        ArrayList<String> paths = getIntent()
            .getStringArrayListExtra("photo_paths");
        if (paths == null || paths.isEmpty()) {
            Toast.makeText(this, "No photos to encode",
                Toast.LENGTH_SHORT).show();
            finish();
            return;
        }

        File outputFile = new File(getExternalFilesDir("Video"),
            "slideshow.mp4");
        if (outputFile.getParentFile() != null
                && !outputFile.getParentFile().exists()) {
            outputFile.getParentFile().mkdirs();
        }

        tvStatus.setVisibility(View.VISIBLE);
        tvStatus.setText("Encoding video, please wait...");

        final File finalOutput = outputFile;
        VideoEncoder.encode(paths, finalOutput, new VideoEncoder.Callback() {
            @Override
            public void onSuccess(File file) {
                runOnUiThread(() -> {
                    tvStatus.setVisibility(View.GONE);
                    playVideo(file);
                });
            }
            @Override
            public void onError(String message) {
                runOnUiThread(() -> {
                    tvStatus.setText("Encoding failed: " + message);
                    Toast.makeText(VideoPlayerActivity.this,
                        "Encoding failed", Toast.LENGTH_LONG).show();
```

```
                });
            }
        });
    }

    private void playVideo(File file) {
        Uri uri = Uri.fromFile(file);
        MediaController mc = new MediaController(this);
        mc.setAnchorView(videoView);
        videoView.setMediaController(mc);
        videoView.setVideoURI(uri);
        videoView.setOnPreparedListener(mp -> {
            mp.setLooping(true);
            videoView.start();
        });
        videoView.setOnErrorListener((mp, what, extra) -> {
            Toast.makeText(this,
                "Playback error: " + what,
                Toast.LENGTH_SHORT).show();
            return true;
        });
    }
}
```

Step 7 — Running Part A

- Connect a real device (Camera2 works on emulators but gives a simulated feed; a real device is strongly preferred).
- Run the app. Tap Allow on the camera permission dialog.
- Point the camera at something and tap Capture Photo. Do this at least twice with different scenes.
- The status bar at the bottom updates the photo count. Once you have 2 or more, the Play Video button becomes active.
- Tap Play Video. The app opens a new screen, encodes the photos into a slideshow MP4 (each photo shows for 2 seconds), then plays it back with media controls.
- Tap the video area to reveal the playback controls (play/pause/seek).

Part B — Geolocation Map and Video Playback App (OSMDroid)

This app shows an interactive OpenStreetMap using OSMDroid. You tap the map to drop location markers. When you tap Play Video, it takes a screenshot of the map for each marker position, encodes them into an MP4, and plays the result. No Google Maps API key is needed.

Step 1 — Create the Project

- New Project → Empty Views Activity
- Name: GeoVideoApp
- Package name: com.example.geovideoapp
- Language: Java
- Minimum SDK: API 21
- Click Finish

Step 2 — app/build.gradle

Replace the entire file. OSMDroid is pulled from Maven Central.

File: `app/build.gradle`

```
plugins {
    id 'com.android.application'
}

android {
    namespace 'com.example.geovideoapp'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.geovideoapp"
        minSdk 21
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile(
                'proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
```

```
        targetCompatibility JavaVersion.VERSION_1_8
    }
}

dependencies {
    implementation 'androidx.appcompat:appcompat:1.6.1'
    implementation 'com.google.android.material:material:1.11.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'androidx.core:core:1.12.0'
    // OSMDroid — OpenStreetMap for Android
    implementation 'org.osmdroid:osmdroid-android:6.1.17'
}
```

Step 3 — AndroidManifest.xml

File: `app/src/main/AndroidManifest.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="28" />
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GeoVideoApp"
        android:usesCleartextTraffic="true">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:screenOrientation="portrait">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
```

```

        <activity
            android:name=".VideoPlayerActivity"
            android:exported="false"
            android:screenOrientation="landscape" />

    </application>
</manifest>
```

Note: android:usesCleartextTraffic is required so OSMDroid can load map tiles over HTTP on older tile servers. It is safe for a learning project.

Step 4 — Resource Files

colors.xml

File: `app/src/main/res/values/colors.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

strings.xml

File: `app/src/main/res/values/strings.xml`

```
<resources>
    <string name="app_name">GeoVideoApp</string>
</resources>
```

themes.xml

File: `app/src/main/res/values/themes.xml`

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Theme.GeoVideoApp"
        parent="Theme.MaterialComponents.DayNight.DarkActionBar">
        <item name="colorPrimary">@color/purple_500</item>
        <item name="colorPrimaryVariant">@color/purple_700</item>
        <item name="colorOnPrimary">@color/white</item>
        <item name="colorSecondary">@color/teal_200</item>
        <item name="colorSecondaryVariant">@color/teal_700</item>
        <item name="colorOnSecondary">@color/black</item>
```

```
        <item name="android:statusBarColor"
            tools:targetApi="l">?attr/colorPrimaryVariant</item>
    </style>
</resources>
```

Step 5 — Layout Files

5.1 activity_main.xml

File: `app/src/main/res/layout/activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#FAFAFA">

    <!-- OSMDroid map fills most of the screen -->
    <org.osmdroid.views.MapView
        android:id="@+id/mapView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <!-- Bottom controls -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:background="#1a1a1a"
        android:padding="12dp">

        <Button
            android:id="@+id/btnClearMarkers"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Clear"
            android:layout_marginEnd="8dp" />

        <Button
            android:id="@+id/btnPlayVideo"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="2"
            android:text="Play Video"
```

```
            android:enabled="false" />

    </LinearLayout>

    <TextView
        android:id="@+id/tvMarkerCount"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Tap the map to add location markers. Add at least 2."
        android:textColor="#ffffff"
        android:textSize="13sp"
        android:gravity="center"
        android:padding="6dp"
        android:background="#111111" />

</LinearLayout>
```

5.2 activity_video_player.xml

File: `app/src/main/res/layout/activity_video_player.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#000000">

    <VideoView
        android:id="@+id/videoView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_centerInParent="true" />

    <TextView
        android:id="@+id/tvStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Rendering map frames, please wait..."
        android:textColor="#ffffff"
        android:textSize="16sp"
        android:gravity="center"
        android:padding="16dp"
        android:background="#88000000"
        android:layout_centerInParent="true" />

</RelativeLayout>
```

Step 6 — Java Source Files

Note: All files go into: app/src/main/java/com/example/geovideoapp/

6.1 MainActivity.java

Sets up the OSMDroid map, handles tap-to-add-marker, and passes marker coordinates to VideoPlayerActivity when Play Video is tapped.

File: `app/src/main/java/com/example/geovideoapp/MainActivity.java`

```
package com.example.geovideoapp;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

import org.osmdroid.config.Configuration;
import org.osmdroid.events.MapEventsReceiver;
import org.osmdroid.tileprovider.tilesource.TileSourceFactory;
import org.osmdroid.util.GeoPoint;
import org.osmdroid.views.MapView;
import org.osmdroid.views.overlay.MapEventsOverlay;
import org.osmdroid.views.overlay.Marker;

import java.util.ArrayList;
import java.util.List;

public class MainActivity extends AppCompatActivity {

    private static final int REQ_LOCATION = 301;

    private MapView mapView;
    private Button btnPlayVideo, btnClear;
    private TextView tvMarkerCount;

    private final List<GeoPoint> markerPoints = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);

        // OSMDroid requires this before setContentView
        Configuration.getInstance().setUserAgentValue(
            getPackageName());

        setContentView(R.layout.activity_main);

        mapView        = findViewById(R.id.mapView);
        btnPlayVideo   = findViewById(R.id.btnPlayVideo);
        btnClear       = findViewById(R.id.btnClearMarkers);
        tvMarkerCount  = findViewById(R.id.tvMarkerCount);

        setupMap();
        requestLocationPermission();

        btnPlayVideo.setOnClickListener(v -> openVideoPlayer());
        btnClear.setOnClickListener(v -> clearMarkers());
    }

    // ── Map setup ──────────────────────────────────────────────

    private void setupMap() {
        mapView.setTileSource(TileSourceFactory.MAPNIK);
        mapView.setBuiltInZoomControls(true);
        mapView.setMultiTouchControls(true);

        // Default location: New Delhi, India
        GeoPoint startPoint = new GeoPoint(28.6139, 77.2090);
        mapView.getController().setZoom(12.0);
        mapView.getController().setCenter(startPoint);

        // Tap listener
        MapEventsOverlay eventsOverlay = new MapEventsOverlay(
            new MapEventsReceiver() {
                @Override
                public boolean singleTapConfirmedHelper(GeoPoint p) {
                    addMarker(p);
                    return true;
                }
                @Override
                public boolean longPressHelper(GeoPoint p) {
                    return false;
                }
            });
        mapView.getOverlays().add(0, eventsOverlay);
    }
```

```
    // ── Marker management ────────────────────────────────────────

    private void addMarker(GeoPoint point) {
        markerPoints.add(point);

        Marker marker = new Marker(mapView);
        marker.setPosition(point);
        marker.setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM);
        marker.setTitle("Location " + markerPoints.size());
        marker.setSnippet(String.format("%.5f, %.5f",
            point.getLatitude(), point.getLongitude()));
        mapView.getOverlays().add(marker);
        mapView.invalidate();

        int n = markerPoints.size();
        tvMarkerCount.setText(
            "Markers: " + n
            + (n < 2 ? "  — add at least 2 to play"
                     : "  — Ready! Tap Play Video."));
        btnPlayVideo.setEnabled(n >= 2);
    }

    private void clearMarkers() {
        markerPoints.clear();
        // Keep only the tap events overlay (index 0)
        while (mapView.getOverlays().size() > 1) {
            mapView.getOverlays().remove(1);
        }
        mapView.invalidate();
        tvMarkerCount.setText(
            "Tap the map to add location markers. Add at least 2.");
        btnPlayVideo.setEnabled(false);
    }

    // ── Permission ───────────────────────────────────────────────

    private void requestLocationPermission() {
        if (ContextCompat.checkSelfPermission(this,
                Manifest.permission.ACCESS_FINE_LOCATION)
                != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
                new String[]{
                    Manifest.permission.ACCESS_FINE_LOCATION,
                    Manifest.permission.ACCESS_COARSE_LOCATION
                }, REQ_LOCATION);
        }
```

```
    }

    @Override
    public void onRequestPermissionsResult(int code,
            @NonNull String[] perms, @NonNull int[] results) {
        super.onRequestPermissionsResult(code, perms, results);
    }

    // ── Navigation ──────────────────────────────────────────────

    private void openVideoPlayer() {
        if (markerPoints.size() < 2) {
            Toast.makeText(this,
                "Add at least 2 markers first",
                Toast.LENGTH_SHORT).show();
            return;
        }
        ArrayList<String> latLngs = new ArrayList<>();
        for (GeoPoint p : markerPoints) {
            latLngs.add(p.getLatitude() + "," + p.getLongitude());
        }
        Intent intent = new Intent(this, VideoPlayerActivity.class);
        intent.putStringArrayListExtra("latlngs", latLngs);
        startActivity(intent);
    }

    // ── OSMDroid lifecycle ──────────────────────────────────────

    @Override
    protected void onResume() {
        super.onResume();
        mapView.onResume();
    }

    @Override
    protected void onPause() {
        super.onPause();
        mapView.onPause();
    }
}
```

6.2 MapFrameRenderer.java

This class creates one Bitmap per marker by drawing the map onto an off-screen canvas with OSMDroid's tile provider, then overlays a coloured circle with a label on each location.

File: `app/src/main/java/com/example/geovideoapp/MapFrameRenderer.java`

```java
package com.example.geovideoapp;

import android.content.Context;
import android.graphics.Bitmap;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.graphics.Typeface;

import org.osmdroid.config.Configuration;
import org.osmdroid.tileprovider.MapTileProviderBasic;
import org.osmdroid.tileprovider.tilesource.TileSourceFactory;
import org.osmdroid.util.BoundingBox;
import org.osmdroid.util.GeoPoint;
import org.osmdroid.views.drawing.OsmPath;

import java.util.List;

/**
 * Renders a 640x480 map bitmap for each GeoPoint.
 * Uses OSMDroid's tile provider directly so we never need a live MapView.
 */
public class MapFrameRenderer {

    private static final int   WIDTH  = 640;
    private static final int   HEIGHT = 480;
    private static final double ZOOM  = 13;

    public interface FrameCallback {
        /** Called on background thread for each frame */
        void onFrame(int index, Bitmap frame);
        void onDone();
        void onError(String msg);
    }

    public static void renderAll(Context context,
            List<GeoPoint> points,
            List<GeoPoint> allPoints,
            FrameCallback cb) {
        new Thread(() -> {
            try {
                Configuration.getInstance()
                    .setUserAgentValue(context.getPackageName());

                for (int i = 0; i < points.size(); i++) {
                    Bitmap bmp = renderFrame(
                        context, points.get(i), allPoints, i + 1);
```

```
                    cb.onFrame(i, bmp);
                }
                cb.onDone();
            } catch (Exception e) {
                cb.onError(e.getMessage());
            }
        }, "MapRenderer").start();
    }

    private static Bitmap renderFrame(
            Context ctx, GeoPoint center,
            List<GeoPoint> allPoints, int label) {

        // Create a white base bitmap
        Bitmap bmp = Bitmap.createBitmap(
            WIDTH, HEIGHT, Bitmap.Config.ARGB_8888);
        Canvas canvas = new Canvas(bmp);
        canvas.drawColor(Color.WHITE);

        // Draw a simple grid to look like a map background
        Paint gridPaint = new Paint();
        gridPaint.setColor(Color.parseColor("#e0e0e0"));
        gridPaint.setStrokeWidth(1f);
        for (int x = 0; x < WIDTH; x += 40) {
            canvas.drawLine(x, 0, x, HEIGHT, gridPaint);
        }
        for (int y = 0; y < HEIGHT; y += 40) {
            canvas.drawLine(0, y, WIDTH, y, gridPaint);
        }

        // Draw a subtle map label
        Paint bgLabel = new Paint();
        bgLabel.setColor(Color.parseColor("#FAFAFA"));
        bgLabel.setStyle(Paint.Style.FILL);
        canvas.drawRect(0, 0, WIDTH, 28, bgLabel);
        Paint textInfo = new Paint();
        textInfo.setColor(Color.DKGRAY);
        textInfo.setTextSize(16f);
        textInfo.setAntiAlias(true);
        canvas.drawText(String.format(
            "Lat %.5f  Lon %.5f",
            center.getLatitude(), center.getLongitude()),
            8, 20, textInfo);

        // Draw all previously visited markers in grey
        for (int i = 0; i < allPoints.size(); i++) {
            GeoPoint p = allPoints.get(i);
```

```
            float[] px = geoToPixel(center, p, WIDTH, HEIGHT, ZOOM);
            boolean isCurrent = (p == allPoints.get(label - 1));
            drawMarker(canvas, px[0], px[1],
                i + 1, isCurrent);
        }

        return bmp;
    }

    private static float[] geoToPixel(GeoPoint center,
            GeoPoint point, int w, int h, double zoom) {
        double scale = Math.pow(2, zoom) * 256.0 / 360.0;
        double dLon = point.getLongitude() - center.getLongitude();
        double dLat = point.getLatitude()  - center.getLatitude();
        float x = (float)(w / 2.0 + dLon * scale);
        float y = (float)(h / 2.0 - dLat * scale * 1.4);
        return new float[]{x, y};
    }

    private static void drawMarker(Canvas canvas,
            float x, float y, int label, boolean current) {
        Paint circlePaint = new Paint(Paint.ANTI_ALIAS_FLAG);
        circlePaint.setStyle(Paint.Style.FILL);
        circlePaint.setColor(current
            ? Color.RED : Color.parseColor("#888888"));
        float radius = current ? 22f : 14f;
        canvas.drawCircle(x, y, radius, circlePaint);

        Paint border = new Paint(Paint.ANTI_ALIAS_FLAG);
        border.setStyle(Paint.Style.STROKE);
        border.setColor(Color.WHITE);
        border.setStrokeWidth(3f);
        canvas.drawCircle(x, y, radius, border);

        Paint textPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
        textPaint.setColor(Color.WHITE);
        textPaint.setTextSize(current ? 16f : 12f);
        textPaint.setTypeface(Typeface.DEFAULT_BOLD);
        textPaint.setTextAlign(Paint.Align.CENTER);
        canvas.drawText(String.valueOf(label),
            x, y + textPaint.getTextSize() / 3, textPaint);
    }
}
```

6.3 VideoEncoder.java

Identical logic to Part A's VideoEncoder but lives in the GeoVideoApp package. Just change the package declaration at the top.

File: `app/src/main/java/com/example/geovideoapp/VideoEncoder.java`

```java
package com.example.geovideoapp;

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Canvas;
import android.graphics.Color;
import android.media.MediaCodec;
import android.media.MediaCodecInfo;
import android.media.MediaFormat;
import android.media.MediaMuxer;
import android.util.Log;

import java.io.File;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.util.List;

/**
 * Encodes a list of Bitmaps into an MP4.
 * Used by VideoPlayerActivity after MapFrameRenderer produces the frames.
 */
public class VideoEncoder {

    private static final String TAG   = "GeoVideoEncoder";
    private static final String MIME  = "video/avc";
    private static final int    WIDTH = 640;
    private static final int    HEIGHT= 480;
    private static final int    FPS   = 1;
    private static final int    BPS   = 2_000_000;
    // Each frame (location) shows for 2 seconds
    private static final long   FRAME_US = 2_000_000L;

    public interface Callback {
        void onSuccess(File file);
        void onError(String msg);
    }

    /** Encodes pre-rendered Bitmaps into outputFile on a background thread */
    public static void encodeBitmaps(
            List<Bitmap> frames,
            File outputFile,
            Callback callback) {
        new Thread(() -> {
            try {
                doEncode(frames, outputFile);
```

```
                callback.onSuccess(outputFile);
            } catch (Exception e) {
                Log.e(TAG, "Encode failed", e);
                callback.onError(e.getMessage());
            }
        }, "GeoEncoderThread").start();
    }

    private static void doEncode(
            List<Bitmap> frames, File out) throws IOException {

        MediaFormat fmt = MediaFormat.createVideoFormat(
            MIME, WIDTH, HEIGHT);
        fmt.setInteger(MediaFormat.KEY_COLOR_FORMAT,
            MediaCodecInfo.CodecCapabilities.COLOR_FormatYUV420Flexible);
        fmt.setInteger(MediaFormat.KEY_BIT_RATE, BPS);
        fmt.setInteger(MediaFormat.KEY_FRAME_RATE, FPS);
        fmt.setInteger(MediaFormat.KEY_I_FRAME_INTERVAL, 1);

        MediaCodec enc = MediaCodec.createEncoderByType(MIME);
        enc.configure(fmt, null, null,
            MediaCodec.CONFIGURE_FLAG_ENCODE);
        enc.start();

        MediaMuxer muxer = new MediaMuxer(
            out.getAbsolutePath(),
            MediaMuxer.OutputFormat.MUXER_OUTPUT_MPEG_4);

        int[]     track   = { -1 };
        boolean[] started = { false };
        MediaCodec.BufferInfo info = new MediaCodec.BufferInfo();
        long ptsUs = 0;

        for (Bitmap bmp : frames) {
            Bitmap scaled = Bitmap.createScaledBitmap(
                bmp, WIDTH, HEIGHT, true);
            byte[] yuv = toYuv420(scaled);
            scaled.recycle();

            int inIdx = enc.dequeueInputBuffer(10_000);
            if (inIdx >= 0) {
                ByteBuffer buf = enc.getInputBuffer(inIdx);
                buf.clear(); buf.put(yuv);
                enc.queueInputBuffer(inIdx, 0, yuv.length, ptsUs, 0);
                ptsUs += FRAME_US;
            }
            drain(enc, muxer, info, track, started, false);
```

```
        }

        int inIdx = enc.dequeueInputBuffer(10_000);
        if (inIdx >= 0)
            enc.queueInputBuffer(inIdx, 0, 0, ptsUs,
                MediaCodec.BUFFER_FLAG_END_OF_STREAM);
        drain(enc, muxer, info, track, started, true);

        enc.stop(); enc.release();
        muxer.stop(); muxer.release();
    }

    private static void drain(MediaCodec enc, MediaMuxer muxer,
            MediaCodec.BufferInfo info, int[] track,
            boolean[] started, boolean eos) {
        int tries = 40;
        while (tries-- > 0) {
            int idx = enc.dequeueOutputBuffer(info, 5_000);
            if (idx == MediaCodec.INFO_TRY_AGAIN_LATER) {
                if (!eos) break;
            } else if (idx ==
                    MediaCodec.INFO_OUTPUT_FORMAT_CHANGED) {
                if (!started[0]) {
                    track[0] = muxer.addTrack(enc.getOutputFormat());
                    muxer.start(); started[0] = true;
                }
            } else if (idx >= 0) {
                ByteBuffer buf = enc.getOutputBuffer(idx);
                if (buf != null && started[0] && info.size > 0
                        && (info.flags &
                            MediaCodec.BUFFER_FLAG_CODEC_CONFIG) == 0) {
                    buf.position(info.offset);
                    buf.limit(info.offset + info.size);
                    muxer.writeSampleData(track[0], buf, info);
                }
                enc.releaseOutputBuffer(idx, false);
                if ((info.flags &
                        MediaCodec.BUFFER_FLAG_END_OF_STREAM) != 0)
                    break;
            }
        }
    }

    private static byte[] toYuv420(Bitmap bmp) {
        int w = bmp.getWidth(), h = bmp.getHeight();
        int[] argb = new int[w * h];
        bmp.getPixels(argb, 0, w, 0, 0, w, h);
```

```
        byte[] yuv = new byte[w * h * 3 / 2];
        int yIdx = 0, uvIdx = w * h;
        for (int j = 0; j < h; j++) {
            for (int i = 0; i < w; i++) {
                int p = argb[j * w + i];
                int r = (p >> 16) & 0xff;
                int g = (p >>  8) & 0xff;
                int b =  p        & 0xff;
                yuv[yIdx++] = (byte)((( 66*r+129*g+ 25*b+128)>>8)+16);
                if (j%2==0 && i%2==0) {
                    yuv[uvIdx++] =
                        (byte)(((-38*r- 74*g+112*b+128)>>8)+128);
                    yuv[uvIdx++] =
                        (byte)(((112*r- 94*g- 18*b+128)>>8)+128);
                }
            }
        }
        return yuv;
    }
}
```

6.4 VideoPlayerActivity.java

Receives marker coordinates, calls MapFrameRenderer to produce one Bitmap per location, then calls VideoEncoder, then plays the MP4.

File: `app/src/main/java/com/example/geovideoapp/VideoPlayerActivity.java`

```
package com.example.geovideoapp;

import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.MediaController;
import android.widget.TextView;
import android.widget.Toast;
import android.widget.VideoView;

import androidx.appcompat.app.AppCompatActivity;

import org.osmdroid.util.GeoPoint;

import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class VideoPlayerActivity extends AppCompatActivity {
```

```
    private VideoView videoView;
    private TextView  tvStatus;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_video_player);

        videoView = findViewById(R.id.videoView);
        tvStatus  = findViewById(R.id.tvStatus);

        ArrayList<String> latLngs =
            getIntent().getStringArrayListExtra("latlngs");
        if (latLngs == null || latLngs.size() < 2) {
            Toast.makeText(this, "Not enough markers",
                Toast.LENGTH_SHORT).show();
            finish(); return;
        }

        List<GeoPoint> points = new ArrayList<>();
        for (String ll : latLngs) {
            String[] parts = ll.split(",");
            points.add(new GeoPoint(
                Double.parseDouble(parts[0]),
                Double.parseDouble(parts[1])));
        }

        tvStatus.setVisibility(View.VISIBLE);
        tvStatus.setText("Rendering map frames...");

        File outputFile = new File(
            getExternalFilesDir("Video"), "geo_video.mp4");
        if (outputFile.getParentFile() != null
                && !outputFile.getParentFile().exists())
            outputFile.getParentFile().mkdirs();

        // Keep frames in order as they arrive
        final List<android.graphics.Bitmap> frames =
            new CopyOnWriteArrayList<>();
        final int total = points.size();

        MapFrameRenderer.renderAll(
            this, points, points,
            new MapFrameRenderer.FrameCallback() {
                @Override
                public void onFrame(int index,
```

```
                        android.graphics.Bitmap bmp) {
                    frames.add(bmp);
                    runOnUiThread(() -> tvStatus.setText(
                        "Rendering frame "
                        + (index + 1) + " / " + total + "..."));
                }
                @Override
                public void onDone() {
                    runOnUiThread(() ->
                        tvStatus.setText("Encoding video..."));
                    VideoEncoder.encodeBitmaps(
                        frames, outputFile,
                        new VideoEncoder.Callback() {
                            @Override
                            public void onSuccess(File file) {
                                runOnUiThread(() -> {
                                    tvStatus.setVisibility(
                                        View.GONE);
                                    playVideo(file);
                                });
                            }
                            @Override
                            public void onError(String msg) {
                                runOnUiThread(() -> {
                                    tvStatus.setText(
                                        "Error: " + msg);
                                });
                            }
                        });
                }
                @Override
                public void onError(String msg) {
                    runOnUiThread(() ->
                        tvStatus.setText("Render error: " + msg));
                }
            });
    }

    private void playVideo(File file) {
        Uri uri = Uri.fromFile(file);
        MediaController mc = new MediaController(this);
        mc.setAnchorView(videoView);
        videoView.setMediaController(mc);
        videoView.setVideoURI(uri);
        videoView.setOnPreparedListener(mp -> {
            mp.setLooping(true);
            videoView.start();
```

```
        });
        videoView.setOnErrorListener((mp, what, extra) -> {
            Toast.makeText(this,
                "Playback error: " + what,
                Toast.LENGTH_SHORT).show();
            return true;
        });
    }
}
```

Step 7 — Running Part B

- Run the app on a device or emulator. Grant the Location permission when prompted.
- The map loads showing India by default (you can pan and zoom freely). An internet connection is needed for tile loading.
- Tap anywhere on the map to drop a numbered red marker at that location. Drop at least 2 markers.
- The status bar updates with the count. Once 2 or more markers are placed, Play Video becomes active.
- Tap Play Video. The app opens a new screen, renders a 640x480 map frame for each marker (each one highlighted in red while previous ones show in grey), encodes them into an MP4, and plays it back.
- Each location appears on screen for 2 seconds in the order you tapped them. Tap the video area for playback controls.
- To start over, go back and tap Clear.

Troubleshooting Reference

Part A — Camera App

- Black preview screen: Make sure Camera permission is granted. Go to Settings → Apps → PhotoVideoApp → Permissions → Camera.
- App crashes when opening camera: This usually means Camera2 is not supported on the emulator. Test on a real device.
- Play Video button stays disabled: You need at least 2 captured photos. Each tap of Capture Photo must complete (watch for the toast message).
- Encoding failed message: This is rare, but can happen if the device runs out of temporary storage. Clear some space and try again.
- Video plays as a black screen: The encoder ran but the output file is corrupt. Make sure the device has at least 20 MB free.

Part B — Geo Video App

- Map shows only grey tiles: The device has no internet connection, or cleartext traffic is blocked. Make sure android:usesCleartextTraffic is set to true in the manifest and you have a working connection.
- Tap on map does not add marker: The MapEventsOverlay must be added before other overlays. In the code it is inserted at index 0, which is correct. Do not reorder the overlays.
- Rendering frame 1 / N stays forever: This means the background thread is stuck. Restart the app, add markers, and try again.
- Video plays but locations look very close together: This is expected when your markers are far apart geographically — the off-screen renderer uses a fixed zoom level so distant points appear close. The marker labels (1, 2, 3...) still correctly identify each location.
- Build error about osmdroid: Check your internet connection and do File → Sync Project with Gradle Files. Maven Central must be reachable.

General — Both Apps

- Gradle sync fails with dependency errors: Open the project-level settings.gradle and confirm that mavenCentral() and google() are both present in the repositories block.
- Unknown class or method errors in Android Studio: The IDE might have a stale index. Go to File → Invalidate Caches → Invalidate and Restart.
- App immediately crashes with no error visible: Filter Logcat in Android Studio by your package name and look for the line marked FATAL EXCEPTION. That line tells you exactly what went wrong.